AgriGraph AI

Used Technologies and Simplified STGNN Pipeline

Abstract

This note lists the main software used in the AgriGraph project and outlines the spatio-temporal graph neural network (STGNN) stage at a high level, without full mathematical or implementation detail.

1 Used technologies

1.1 Backend and machine learning

The Python backend relies on:

- **FastAPI** and **Uvicorn** for the HTTP API and ASGI server.
- **NumPy**, **pandas**, and **SciPy** for numerical work and tabular data.
- **scikit-learn** for preprocessing (e.g., scaling), PCA, and regression metrics.
- **PyTorch** for deep learning (CVAE, STGNN).
- **PyTorch Geometric** (`torch_geometric`), in particular `GCNConv` for graph convolution.

1.2 Frontend

The web client is built with:

- **React** 18 and **React Router** for UI and routing.
- **Vite** as the build tool and dev server.
- **Tailwind CSS** for styling.
- **Recharts** for charts; **react-simple-maps** and **topojson-client** for map visualisation.

2 Simplified end-to-end pipeline (context for STGNN)

County-level herd indicators are cleaned, optionally augmented with a year-conditioned CVAE, compressed into pillar scores via PCA, and embedded in a fixed spatial graph over the 26 Irish counties. The STGNN then learns from multi-year node sequences. Figure 1 summarises how these steps connect.

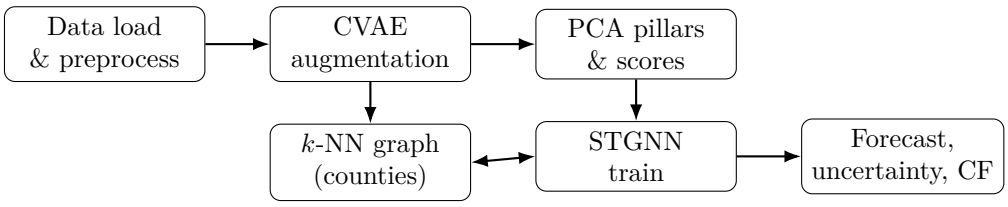


Figure 1: Research pipeline (abbreviated): preprocessing and augmentation lead to pillar scores; a spatial graph is built; the STGNN is trained; downstream modules consume the trained model.

3 Simplified STGNN pipeline

3.1 Graph and inputs

Each *node* is one county (26 nodes). Edges come from geographic proximity: pairwise great-circle distances between county centroids, keeping the k nearest neighbours per county (with symmetrisation as implemented). Node features per year combine four PCA-derived pillar dimensions plus an overall sustainability score (five channels per node).

3.2 Temporal setup

For each data variant (real plus synthetic augmentations), the model sees a short sequence of calendar years, e.g. four input years leading to a target year. The network stacks per-timestep spatial embeddings and applies a *causal* temporal attention mechanism so that prediction at the last timestep only attends to past and present steps (no leakage from future years within the window).

3.3 Spatial–temporal model (conceptual)

1. **Spatial block:** For each time step, two graph convolution layers (`GCNConv`) with ReLU and dropout propagate information along county edges.
2. **Temporal block:** Learned linear projections form queries, keys, and values over the time dimension; attention weights combine past values into a context vector for each county.
3. **Regression head:** A small fully connected network maps the temporal context to a scalar target per county (e.g. sustainability-related outcome for the forecast year).

3.4 Training modes (what they are for)

Two training passes share the same architecture and hyperparameters but differ in data use:

- **Evaluation training** holds out the real target year for honest metrics (e.g. R^2, MAE) on unseen real counties for that year.
- **Full retraining** uses all available observed pairs for the forecasting model that drives projections, uncertainty, and counterfactual views; test metrics from this pass are not interpreted as out-of-sample performance.

Optimizer: Adam with fixed learning rate and weight decay; loss is mean squared error on targets (with regularisation as configured in code).

For column names, county lists, and pillar–feature mappings, see the project data contract and rules in the repository.